ShuffleSnap: strided windowed exact reassignment for point-to-grid layout

Kyle McDonald

October 6, 2026

Abstract

Giving each point of a 2D embedding its own grid cell with the least total squared displacement is a linear assignment problem too large for dense solvers at 10^5 points. ShuffleSnap, inspired by Shellsort, solves small exact assignment problems in windows whose cells lie a stride apart, and shrinks the stride to one. Against certified optima it comes within hundredths of a percent on clustered layouts, including real UMAP embeddings, in seconds. Its errors are loops of one-cell shifts that no window can see, which make uniform points, concave masks and surplus cells hard.

1 From points to cells

An image collection is often shown by embedding the items in the plane, for example with t-SNE or UMAP [6, 10], and placing each thumbnail in its own grid cell. Rounding every point to its nearest cell causes collisions and holes, so the cells must be assigned jointly: we seek the assignment σ of points p_i to distinct cells $c_{\sigma(i)}$ that minimizes $\sum_i \|p_i - c_{\sigma(i)}\|^2$. This linear assignment problem, also a discrete optimal-transport problem [8], is solved exactly by dense solvers [2, 4] that need the full cost matrix: 80 GB at 10^5 points. Practical tools instead split the cloud recursively, like RasterFairy [3], or merge randomized partitions, like BSP-OT [1].

ShuffleSnap borrows from Shellsort [9], which sorts interleaved subsequences at a shrinking gap. It solves only small assignment problems, but exactly, with their cells spread across the grid first and brought together later (Figure 1). We measure its gaps against a solver that proves optimality on the same instances, and report where it fails as well as where it works.

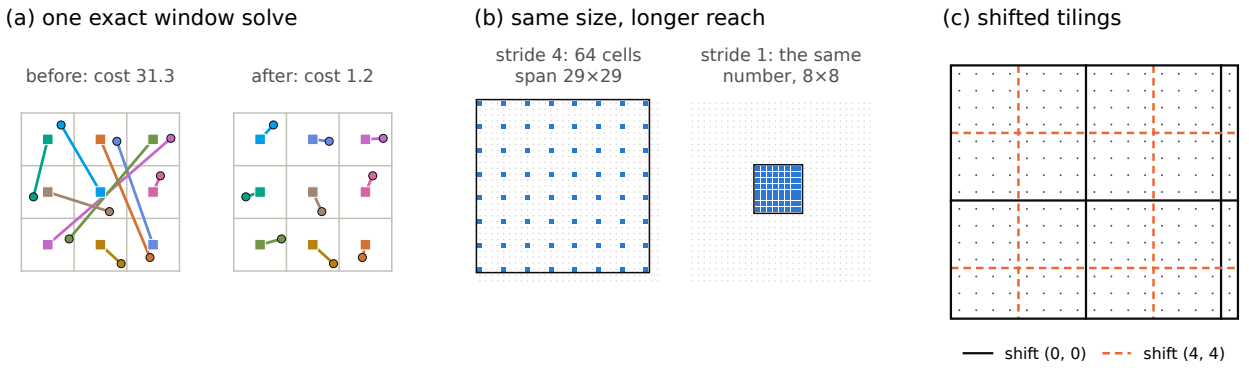


Figure 1: (a) A window collects the points that occupy its cells and reassigns them among the same cells by an exact linear assignment. (b) At stride s a window takes every s-th row and column: it keeps 64 cells but spans $7s + 1$ cells per axis, so points can travel far. Strides shrink from left to right. (c) Each subgrid is tiled four times, shifted by half a window, so that points can cross every window border.

2 Coarse to fine

ShuffleSnap starts from a random assignment. At stride s we keep every s-th row and every s-th column of the grid, which splits it into s^2 interleaved subgrids, one for each choice of first row and first column. Each subgrid is cut into windows of 8×8 of its cells. In every window, the current occupants are reassigned among the window's cells by solving a linear assignment problem of at most 64×64 exactly, with costs taken from the original point coordinates. The new assignment replaces the old one only if it lowers the window's cost by more than a relative 10^{-12}, so the layout stays valid, its cost never increases, and ties cannot cause cycles.

A round tiles every subgrid four times, shifted by $(0,0)$, $(0,4)$, $(4,0)$ and $(4,4)$. Windows that cross an edge are clipped, not dropped. The windows of one tiling are disjoint, so they are solved in parallel. On an axis of length L the first stride is the smallest power of two with $8s \geq L$, which makes the first windows span the grid. After each round the stride is divided by $\sqrt{2}$ and rounded; at stride 1, rounds repeat until one moves no point. Each level costs about the same, linear in the number of cells, and the no-move stop is a fixed point of the windows, not a proof of optimality. We chose the ratio $\sqrt{2}$ experimentally: no ratio between 1.1 and 3 was clearly better. Larger windows ended closer to the optimum in every test, but took longer to reach any cost that a smaller window also reached. Besides the default 8×8 we offer 5×5 (*fast*) and 12×12 (*quality*).

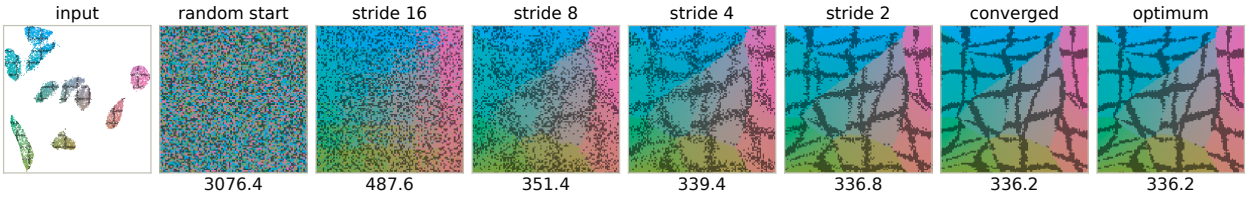


Figure 2: Progress on 10,000 points of a UMAP embedding of MNIST digits [5] (every second stride shown). Every point keeps the color of its input position, and the dark bands are graph-paper lines drawn on the input: both travel with the points. Numbers are the mean squared displacement in cells^2.

Targets and implementation. A target can be any set of cells: a rectangle, a disk, the shape of a letter. Each axis has its own stride, masked cells never enter a window, and empty cells do, so with more cells than points the solver also chooses which cells stay empty. The kernel is C++; a 55-line NumPy version (Appendix C) reproduces it exactly. At coarse strides a window's points often form one distant cluster that all prefer the same corner cell, which stalls generic solvers. Starting each exact solve from closed-form duals of per-axis optimal transport (sorting) leaves results unchanged but makes window solves 2–5 times faster, enough to afford 8×8 windows at BSP-OT's running time.

Certified optima. To measure gaps, we solve a min-cost flow with OR-Tools [7] on sparse candidate edges: each point's 12 nearest cells and the neighbourhood of its cell in a valid starting layout (ShuffleSnap's). As this may miss an edge, we check it against the full, dense problem, whose optimal cost any cell potentials v bound from below:

$$\mathrm{LB}(v) = \sum_i \min_j \big(\|p_i - c_j\|^2 - v_j\big) + \sum_j v_j - (M - N) \max_j v_j,$$

where $M \geq N$ is the number of cells. We derive v from the sparse solution, evaluate the bound over *all* cells, add every edge it shows to be missing, and repeat until the bound meets the cost. Then no assignment of the dense problem is cheaper, whatever the start. It certified every benchmark instance (up to 2×10^5 points); all gaps below use it.

3 Evidence and limits

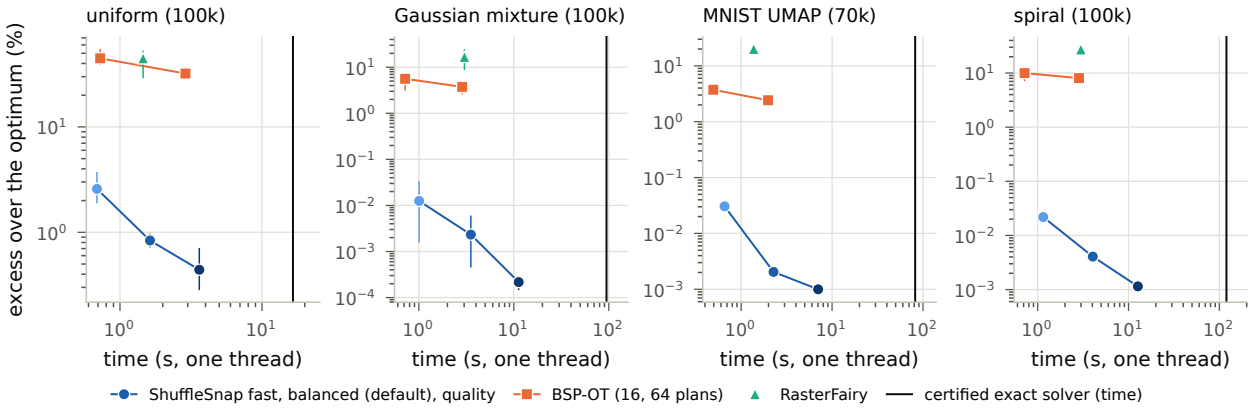


Figure 3: Excess over the certified optimum against time on one core. Points are means over three instances (for MNIST, three runs on the one embedding); bars span their range. The three ShuffleSnap presets use 5×5, 8×8 (default) and 12×12 windows from left to right. The vertical line is the time of the certified exact solver.

We ran ten synthetic distributions with 10^5 points and the complete UMAP embeddings of MNIST and Fashion-MNIST [11] (70,000 points each). All methods received identical inputs, and times include each method's own preparation. Figure 3 shows three and MNIST.

On clustered data, the kind of layout that embeddings produce, ShuffleSnap is close to optimal. It is within 0.06% of the optimum on nine of the ten distributions and takes a median of 2.3 s on one core, or 0.19 s on 24 threads. On the two real embeddings it is within 0.002% of the optimum, and its layouts are visually indistinguishable from the optimal ones (Figure 4). At equal time it beats BSP-OT on most inputs: the fast preset (0.77 s) is closer to the optimum than BSP-OT with 16 plans (0.71 s) on eight of ten distributions, and the default (2.3 s) than 64 plans (2.9 s) on nine; on uniform points they are at 2.6% and 0.84%, against 45% and 32%. RasterFairy is 17–425% above the optimum on seven. Displacement is not the whole story, though: on the real embeddings BSP-OT stays within a few percent of the optimal displacement yet halves the share of grid neighbours that were input neighbours: it scrambles local order.

ShuffleSnap is a local search, and its failures follow from that. Its errors form a few long closed loops in which every point is shifted by about one cell. Such a loop is optimal inside every window that sees only part of it, and a strided window cannot shift a point by one cell. We built inputs on which the method never moves while the optimum costs zero: its relative excess is unbounded. Those inputs need a start the method never chooses. From random starts the result is predictable: on 50 clouds of 10^4 uniform points, 50 random starts each, the excess per point was nearly log-normal and alike on every cloud, with 90% of runs at 0.55–1.38%. On uniform points the loops grow with N: the excess rises from 0.1% at 10^3 points to 0.84% at 10^5 and 1.3% at 10^6. Concave masks and grids with a few surplus cells are harder: with uniform points, 1.4% on a blocky S-shaped mask (BSP-OT: 9.3%) and 3.1% with under a row of surplus cells.

We also ran the default preset on a billion points of a random walk (Appendix B). It converged in 1 h 43 min using 53 GiB; no optimum is known at that size, so this shows that the method scales, not how close it comes. The exact solver took a median of about two minutes at 10^5 points. When a proven optimum is worth that wait, it is the better choice; ShuffleSnap remains the fast answer, a warm start for the exact solver, and the method for larger sizes and tighter budgets. Code: `github.com/kylemcdonald/shufflesnap` (PyPI: `shufflesnap`); results and figure scripts: `github.com/kylemcdonald/shufflesnap-paper`.

References

[1] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Sparse transport plans between discrete measures in loglinear time. *ACM Transactions on Graphics*, 44(6):1–15, December 2025. doi: 10.1145/3763281. URL https://doi.org/10.1145/3763281. Proc. SIGGRAPH Asia 2025.

[2] R. Jonker and A. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. URL https://doi.org/10.1007/BF02278710.

[3] Mario Klingemann. RasterFairy. https://github.com/Quasimondo/RasterFairy, 2015. URL https://github.com/Quasimondo/RasterFairy. GitHub repository created 2016-01-22; latest PyPI release of `rasterfairy` is 1.0.6 (2019-04-29). Accessed 2026-09-23.

[4] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109. URL https://doi.org/10.1002/nav.3800020109.

[5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL https://doi.org/10.1109/5.726791.

[6] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction, 2018. URL https://arxiv.org/abs/1802.03426.

[7] Laurent Perron and Vincent Furnon. OR-Tools. https://developers.google.com/optimization/, 2026. URL https://developers.google.com/optimization/. Version v9.15, released 2026-01-12. Accessed 2026-09-23.

[8] Gabriel Peyré and Marco Cuturi. Computational optimal transport with applications to data sciences. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/2200000073. URL https://doi.org/10.1561/2200000073.

[9] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959. doi: 10.1145/368370.368387. URL https://doi.org/10.1145/368370.368387.

[10] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL https://jmlr.org/papers/v9/vandermaaten08a.html.

[11] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. URL https://arxiv.org/abs/1708.07747.

A Real embeddings

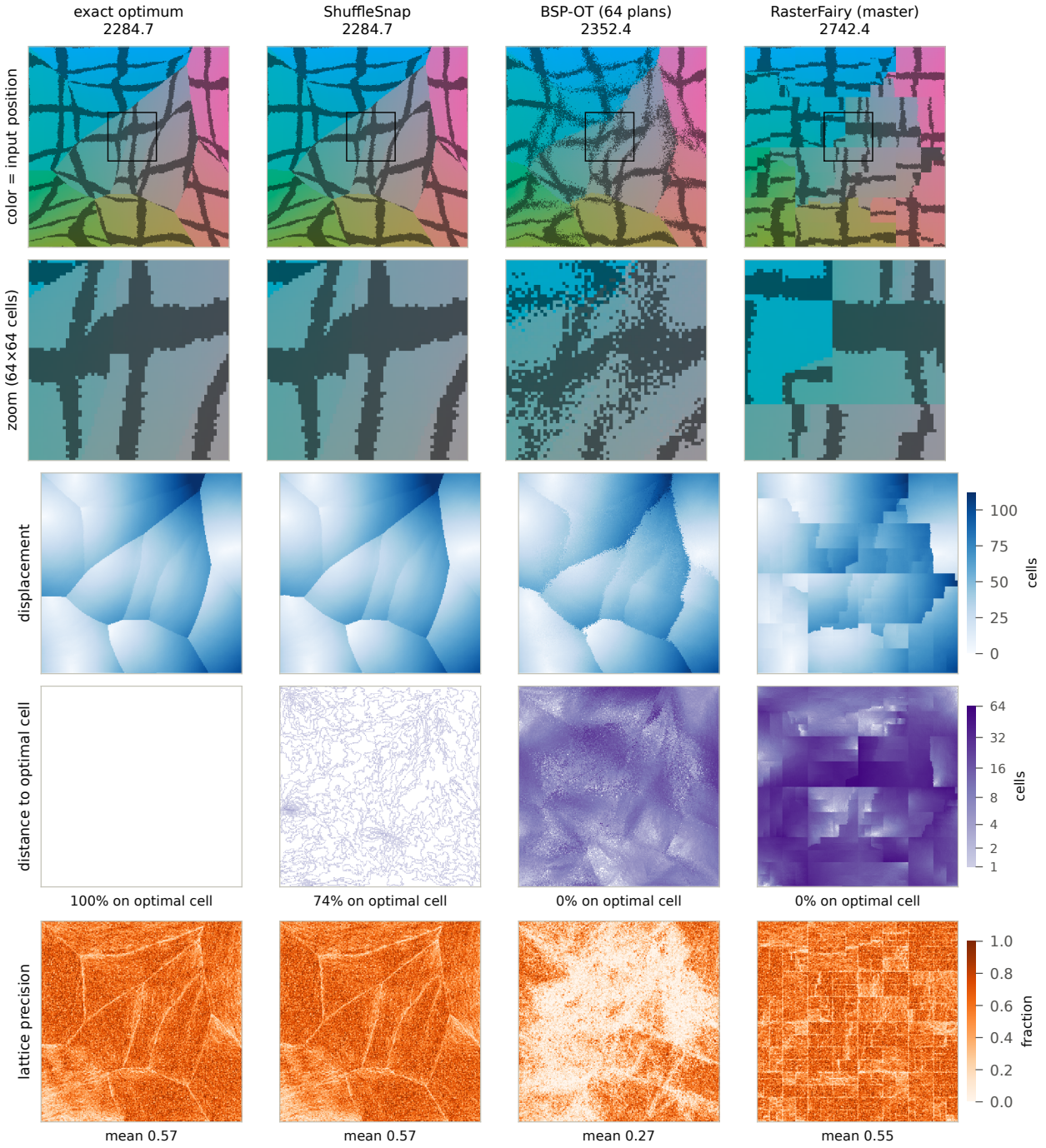


Figure 4: All 70,000 MNIST digits, UMAP layout snapped to a 265×265 grid. Colors and graph-paper lines encode input position as in Figure 2; numbers are the mean squared displacement in cells^2. Rows: the full grid, the 64×64 box enlarged, each point's displacement, its distance to its cell in the certified optimum (log scale; white is the optimal cell), and lattice precision (the fraction of a point's occupied 8-neighbour cells holding one of its 8 nearest input neighbours). ShuffleSnap leaves 74% of the points on their optimal cell, and its errors are thin loops of one-cell shifts. BSP-OT stays within 3% of the optimal displacement, yet its median point is 8 cells from its optimal cell, and it halves lattice precision; RasterFairy breaks the lines at rectangular seams.

B A billion points

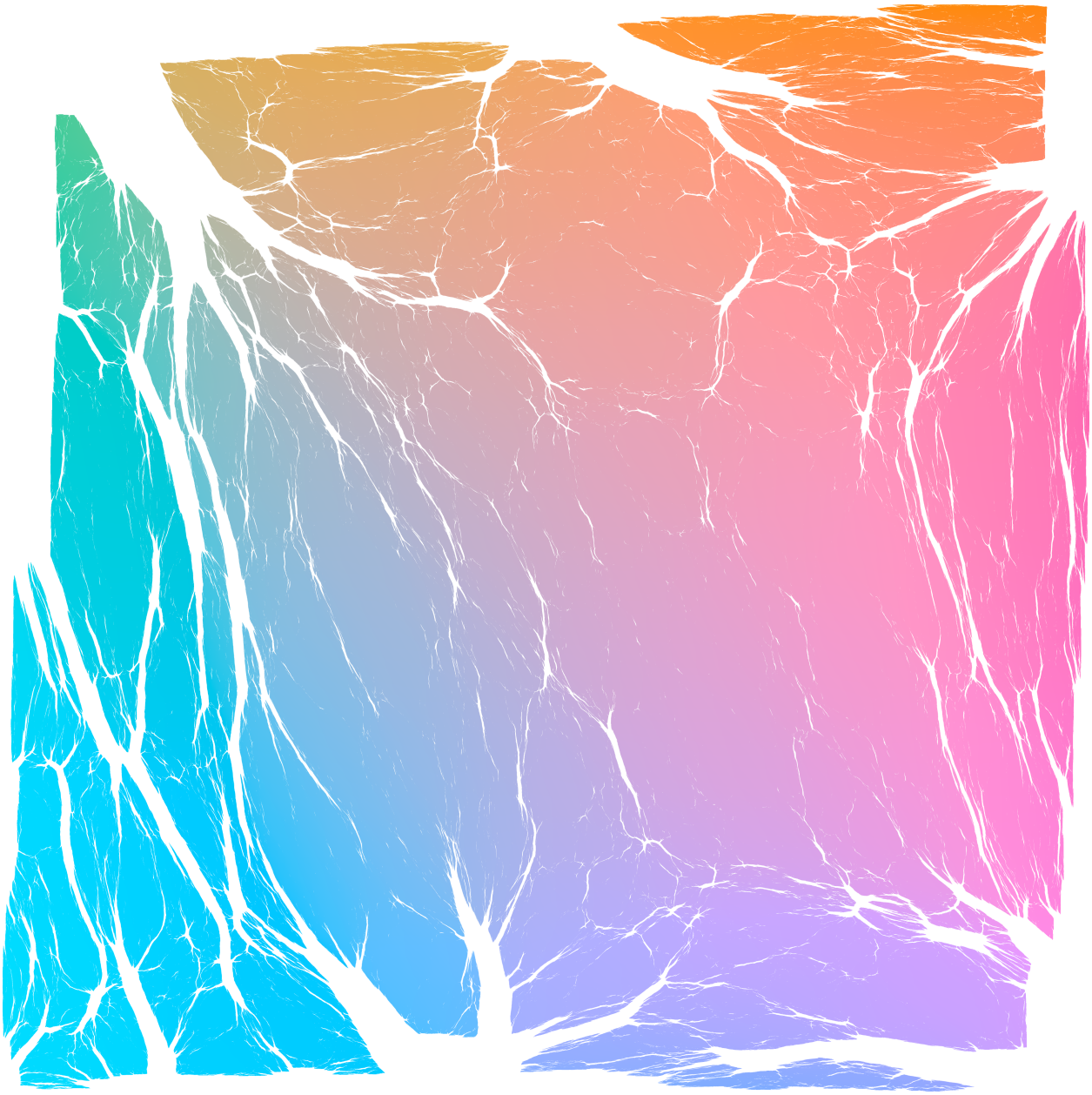
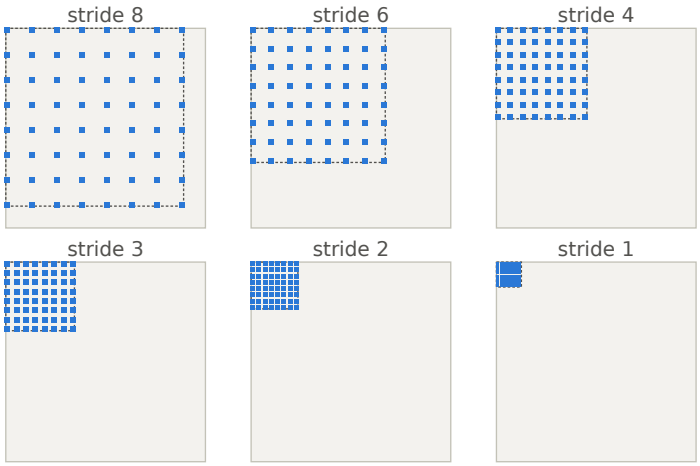


Figure 5: The billion-point run of Section 3: 1,000,014,129 points of a random walk on a 31,623 × 31,623 grid, after the default preset and the quality preset had converged. Every point is drawn 75% of the way from its input position to its assigned cell, colored by input position, and the points are averaged over 10,000 × 10,000 pixels. White seams open where neighbouring cells hold points from distant parts of the walk; they follow its folds.

C A serial reference implementation

This is the complete algorithm: the file `shufflesnap/reference.py` from the package, split into its parts, each beside a diagram computed with its code. Without exact ties it returns the same assignment as the compiled kernel; the package's tests check this for every preset on six kinds of target, from an exact-fit square to surplus cells and a random mask. The kernel adds threads, skips windows whose cells have not changed (which does not change the result) and an optional time budget. Points are in grid units: cell (x, y) has its center at (x, y).

The schedule.



A window always holds $w \times w$ cells, but at stride s they lie s cells apart, so it spans $(w-1)s+1$ cells per axis (dashed). The first stride is the smallest power of two with $ws \geq L$, so the first windows reach across the axis; each later stride is the last one divided by `ratio` and rounded. Each axis has its own strides, and the shorter axis waits at 1. Drawn: one window of the default preset ($w = 8$) on a 64×64 grid at every stride.

```python
"""ShuffleSnap in NumPy and SciPy, written to be read: serial, unit cells.

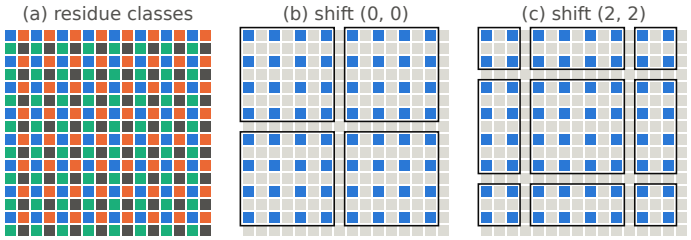
Without exact ties it returns the same cells as the compiled kernel,
``shufflesnap.assign(points, grid, normalize="none", ...)`` (tested).
``snap(P, mask)`` is the default preset; ``w=5`` and ``w=12`` give the
fast and quality presets.
"""
from collections import defaultdict
from itertools import zip_longest

import numpy as np
from scipy.optimize import linear_sum_assignment


def strides(length, w, ratio):
    """Strides on one axis: the first window spans it, then divide by ratio."""
    s = 1
    while w * s < length:
        s *= 2
    out = []
    while s > 1 + 1e-9:
        out.append(max(1, round(s)))
        s /= ratio
    return out


def schedule(width, height, w, ratio):
    """Stride pairs (sx, sy) before stride 1; the shorter axis waits at 1."""
    sx, sy = strides(width, w, ratio), strides(height, w, ratio)
    pairs = []
    for pair in zip_longest(sx, sy, fillvalue=1):
        if pair != (1, 1) and pair not in pairs[-1:]:
            pairs.append(pair)
    return pairs
```

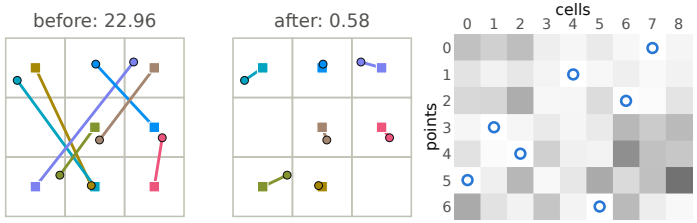
The windows of one tiling.



At stride (s_x, s_y) the cells fall into $s_x s_y$ residue classes ($x \bmod s_x$, $y \bmod s_y$) (a). In its class, cell x is lattice column $x // s_x$, and tiles of w lattice columns start at column `ox`. A window is the set of cells that share a class and a tile (b, boxes). The four tilings of a round shift the tiles by half a window (c). Edge windows are smaller, and masked cells never appear. Drawn with $w = 4$, stride 2, class $(0, 0)$.

```python
def windows(cx, cy, sx, sy, w, ox, oy):
    """One tiling: cells in the same residue class and the same tile."""
    tx = (cx // sx + w - ox) // w  # tiles start at lattice column ox
    ty = (cy // sy + w - oy) // w
    groups = defaultdict(list)
    for cell, key in enumerate(zip(cx % sx, cy % sy, tx, ty)):
        groups[key].append(cell)
    return [np.array(cells) for cells in groups.values()]
```

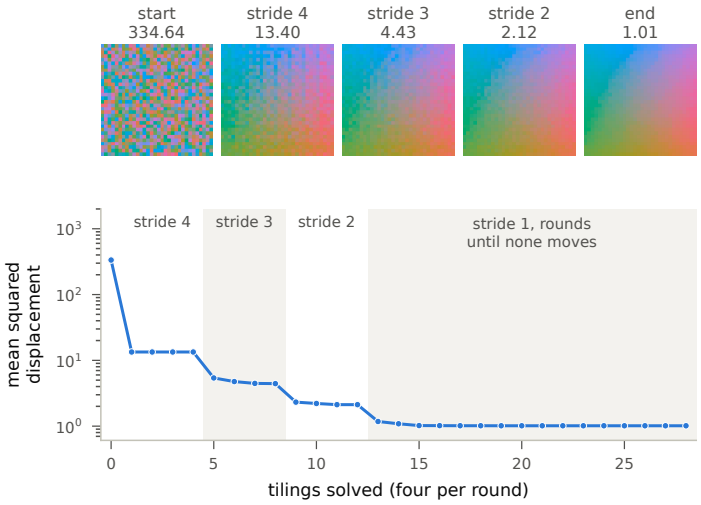
One window.



The window's points are reassigned among all of its cells, empty ones included, by an exact linear assignment on squared distances. The result is kept only if it lowers the window's cost by more than a relative `tol`, so the cost never rises and ties cannot cycle. `pos` gives each point's cell and `occ` each cell's point. Drawn: seven points in a 3×3 window before and after, with summed squared distances, and the cost matrix with the chosen entries.

```python
def solve_window(P, C, cells, occ, pos, tol):
    """Reassign the window's points among its cells; keep it if better."""
    pts = occ[cells][occ[cells] >= 0]  # the points in these cells
    cost = ((P[pts, None] - C[None, cells]) ** 2).sum(-1)  # points x cells
    new = cells[linear_sum_assignment(cost)[1]]
    old_cost = ((P[pts] - C[pos[pts]]) ** 2).sum()
    new_cost = ((P[pts] - C[new]) ** 2).sum()
    if new_cost >= old_cost * (1 - tol):  # strict improvements only
        return False
    occ[cells] = -1
    occ[new] = pts
    pos[pts] = new
    return True
```

The whole method.



The cells are the mask's usable entries in row-major order, and point i starts in a random cell `pos[i]`. A round solves the windows of the four tilings in turn. Each coarse stage runs one round; then stride-1 rounds repeat until one moves nothing, a fixed point of every window but not a proof of optimality. Drawn: 1,024 uniform points on a 32×32 grid, colored by input position, after the start, each coarse stage and the end (top), and the mean squared displacement in cells^2 after every tiling (bottom): each stride halves it or better, and the last stride-1 round moves nothing.

```python
def snap(P, mask, w=8, ratio=2 ** 0.5, seed=0, tol=1e-12):
    """Cell of each point; P is (N, 2) in grid units, cells are row-major."""
    cy, cx = np.nonzero(np.asarray(mask, dtype=bool))
    C = np.stack([cx, cy], axis=1).astype(float)  # cell centers (x, y)
    rng = np.random.default_rng(seed)
    pos = rng.permutation(len(C))[:len(P)]  # random start: cell of point
    occ = np.full(len(C), -1)  # point in each cell, -1 if empty
    occ[pos] = np.arange(len(P))
    h = w // 2

    def one_round(sx, sy):  # four tilings, shifted by half a window
        moved = 0
        for ox, oy in [(0, 0), (0, h), (h, 0), (h, h)]:
            for cells in windows(cx, cy, sx, sy, w, ox, oy):
                moved += solve_window(P, C, cells, occ, pos, tol)
        return moved

    for sx, sy in schedule(mask.shape[1], mask.shape[0], w, ratio):
        one_round(sx, sy)
    while one_round(1, 1):  # stride 1 until nothing moves
        pass
    return pos
```